



AUTOPILOT ASIA HACKATHON 2026 · ROUND 2

Round 2 Guide

Rules, template setup, judging, and the full FAQ, in one document.

Remote build 3 to 7 August · Offline build 8 August · Grand Finale 9 August · Asia Pacific University, Kuala Lumpur

Everything you need for Round 2: the goal of the round, the Orchestrator and Operator model, how to set up and use the template repository, what each Command Center surface has to do, the business functions worth adding to your track, the dataset and integration rules, the full judging gate and rubric, and a long FAQ. Read this fully before you build. Discord is the single source of truth; where a live announcement differs from this document, the announcement is authoritative.

5-day remote build

Agent on Auto + coded Command Center

Gate, then rubric out of 100

0. The one rule that governs everything else

Discord is the single source of truth for Round 2. Every announcement, every schedule change, every office-hours session, and every official ruling from the organizing team and judging panel happens in the official Autopilot Asia Discord server. If a piece of information is not in Discord, it is not official.

Rules published here are the standing framework. Where this document and a live Discord announcement differ, the Discord announcement is authoritative, because it is more recent. The organizing team pins every material ruling so the current state of the rules is always visible in one place.

1. The goal of Round 2

1.1 **In one sentence:** take the AI Employee you built in Round 1, grow it into a wider operation with more Operators, and put a real, coded Command Center around it so a business leader could actually run it.

1.2 Round 1 answered one question: can you decompose a business problem into an Orchestrator and Operators on Supervity Auto, connect real systems, and escalate an exception to a human. Round 2 answers a harder one: **can you run that agent as a governed operation**. Governed means a person sets the rules, the agent works inside them, every decision is visible and auditable, and anything the agent should not decide alone reaches a human with the context to decide it.

1.3 Round 2 does not throw Round 1 away. Your Round 1 Orchestrator and Operators stay as the brain. You extend the agent, and you build the operation around it.

1.4 Every Round 2 build has two layers, and the split between them is mandatory.

Layer one, the agent, runs on Supervity Auto. One Orchestrator coordinating at least five distinct Operator Agents that do the actual domain work. All orchestration must be built on Auto.

Layer two, the Command Center, is coded on the official template. The dashboard, the AI Policies engine, AI Insights, the AI Manager, the Data Manager, and the Workbench are built in the template repository, and you may use any model you like there, including Claude, Gemini, GPT, or open source.

1.5 Teams keep the track they were assigned in Round 1. Tracks were distributed evenly by the Supervity team and do not change between rounds. Team composition does not change either; teams remain one to two people.

2. Orchestrator and Operators

2.1 An **Operator Agent** is a single AI Employee that does one job end to end. It takes an input, does one unit of work, uses the integrations that job needs, and returns a result. One Operator is one skilled worker with one responsibility.

2.2 An **Orchestrator** is an Operator that runs other Operators. It does not do the detailed work itself. It decides which Operators to call and in what order, in sequence, in parallel, or branching on a condition; it passes context and results between them; it retries when one fails; and it escalates to a human at the Auto Workbench when a decision needs one. The Orchestrator is the manager, the Operators are the team.

2.3 **How to call an Operator from an Orchestrator.** In Supervity Auto an Operator is a workflow. From inside the Orchestrator's workflow you call another Operator by naming it in plain language, for example "I want to trigger the Risk Screen Operator Agent" or "call the Extraction operator agent." You state the Operator you want to trigger, and Auto runs that Operator as a step inside the Orchestrator.

2.4 **How to build it, in order.** Build each Operator as its own workflow and get it working on its own first, with its own inputs, its own integrations, and its own output. Then build the Orchestrator as a workflow whose steps trigger those Operator workflows by name, collect their outputs, branch on the result, retry on failure, and escalate exceptions to a human. One giant workflow that does everything is a single mega-agent, and it fails the gate.

2.5 Round 2 raises the floor. The agent must show **at least five distinct Operators** and genuine orchestration depth: parallel fan-out and fan-in, conditional branching, retries, and clean context passing between Operators.

2.6 A useful test of whether your decomposition is real: if you deleted one Operator, would a specific business capability disappear? If the answer is no, you have split one agent for appearances rather than built a team.

3. What you can add in Round 2

3.1 Round 1 covered the core path of each domain. The operation is still missing the business functions around it, and each one is naturally its own Operator. These are suggestions, not requirements. Pick the ones that make your build a fuller AI Employee.

Track	Business functions worth adding
Finance, Accounts Payable	Vendor onboarding and out-of-band bank verification · payment runs that capture early-payment discounts · FX conversion and intercompany allocation · vendor statement reconciliation · dispute and credit-memo handling · month-end accrual and a close pack
Operations, Procurement	Prioritising concurrent disruptions competing for the same stock · RFQ and quote comparison across alternative suppliers · inventory reallocation across warehouses · customer re-promising and comms · penalty and escalation-clause modelling · a rolling supplier scorecard
Customer Support, Service Desk	Major-incident detection and comms · change approval before touching production · rollback and verification when a fix fails · knowledge-base authoring from resolved tickets · CSAT follow-up and escalation · load balancing across teams and on-call
Sales Intelligence	Buying-group resolution across an account · territory and capacity routing with collision handling · regional consent and compliance gating · multi-touch sequence orchestration · reply handling and meeting booking · pipeline hygiene in CRM
HR and People Ops	Multi-jurisdiction compliance deadlines · cross-team provisioning across IT, payroll, facilities and security · first-payroll verification · learning and milestone tracking · attrition risk scoring · manager accountability and escalation

3.2 Adding capability is how a build moves from "it works" to "a business could run this." It is also where bonus points live.

4. The template repository

4.1 Every Round 2 build starts from the official template: <https://github.com/digitamizers/AutoPilot-Template>

4.2 The template is a starter kit, not a finished product. It gives you a working backend, database, and Command Center shell so you spend the five days on the agent and the operation rather than on boilerplate.

What the template already includes

Layer	What you get
Backend	FastAPI with auto-generated Swagger docs, PostgreSQL with Alembic migrations, an auth system with a development bypass, audit-logging middleware on every request, a sample CRUD API, a file-storage API, and a role-based authorization engine.
Frontend	A Next.js and React Command Center: a dashboard with stat cards and an activity chart, an AI Policies page, an AI Insights page, an AI Manager chat interface, a Workbench page, a settings page, and a command palette.
Infrastructure	Docker Compose for one-command startup, a pre-built production frontend, and cross-platform support for macOS, Windows, and Linux.

What you build on top

Surface	Template status	Your task
AI Manager	Chat interface ready	Connect it to your Auto agent through the backend so it can answer from real records and trigger Operators.
AI Policies	Demo data loaded	Build the engine that evaluates real rules at runtime and genuinely constrains the agent.
AI Insights	Demo data loaded	Build the analysis engine that generates insights from the data your agent actually processed.
Workbench	Interface shell ready	Build exception routing, so when the agent pauses, real work items arrive with context and a human resolves them.
Dashboard	Static cards	Wire the KPIs and queues to live agent activity.

4.3 Shipping the template's demo data as if it were your own output is a fail. Judges check the Policies and Insights pages against the data your agent actually processed.

5. Setting up the template

5.1 Prerequisites. Install Docker Desktop and Git. On Windows, enable WSL 2; Docker Desktop will prompt you, and if you see a WSL error run `wsl --install` in PowerShell as Administrator and restart.

5.2 Step one, clone the repository.

```
git clone https://github.com/digitamizers/AutoPilot-Template
cd AutoPilot-Template
```

5.3 Step two, create your environment file. On macOS or Linux run `cp .env.example .env`. On Windows PowerShell run `Copy-Item .env.example .env`. The default file works out of the box: `AUTH_BYPASS=true` means no external auth setup is needed and the app starts with a development session.

5.4 Step three, start Docker Desktop and wait until it reports that it is running. The first launch can take a minute or two.

5.5 Step four, start all services. On macOS or Linux run `make up`. On Windows PowerShell run `.\scripts\start.ps1`, which clears a common WSL 2 port conflict, starts Docker, and verifies the services are reachable. The first run takes two to five minutes to download images and build containers; later runs start in about fifteen seconds.

5.6 Step five, verify. Run `docker compose ps`. You should see three services running: postgres, backend, and frontend.

5.7 Step six, open your Command Center.

Service	URL	What it is
Dashboard	<code>http://localhost:3001</code>	Your Command Center interface
API docs	<code>http://localhost:8001/api/docs</code>	Backend Swagger documentation
Database	<code>localhost:5432</code>	PostgreSQL

5.8 Everyday commands. On macOS and Linux: `make down` stops everything, `make logs-be` and `make logs-fe` stream backend and frontend logs, `make reset-db` resets and re-seeds the database, `make migrate-up` applies migrations, `make lint` lints both sides, and `make test-be` runs backend tests. On Windows use the equivalent `docker compose` commands, for example `docker compose up --build -d`, `docker compose down`, and `docker compose logs -f backend`.

5.9 Read the repository documentation before you build. The template ships with a Command Center guide covering what each surface is meant to do, a design-system reference for UI patterns, and an audit-system guide. Use the design system so your Command Center looks like a Supervity product.

5.10 If something breaks. A port already in use on 3001 or 5432 usually means another service or a local PostgreSQL is running; stop it or change the port in `docker-compose.yml`. Containers that crash-loop are almost always a missing environment variable or a database issue, so check `docker compose logs backend`. A blank frontend usually means the backend is not healthy, so check `curl http://localhost:8001/api/health`. On a first run, give PostgreSQL ten to fifteen seconds to initialize.

5.11 Daily office hours run in Discord throughout the build. Setup problems are the fastest thing to solve there, so ask early rather than losing a day.

6. Connecting the Command Center to your agent

6.1 The Command Center must be genuinely wired to the Auto agent through the backend API. A frontend showing seeded numbers, or a dashboard that never changes when the agent runs, does not pass the gate.

6.2 The flow every build needs: a trigger arrives, the backend calls your Auto Orchestrator, the Orchestrator delegates to its Operators, the proposed action is evaluated against your Policies engine, anything needing a human is routed to the Workbench, the result is executed and persisted, and the dashboard, Insights, and Data Manager reflect what happened.

6.3 **Get your Workflow API key.** Your backend authenticates to Auto with a Workflow API key, which you generate yourself at <https://auto.supervity.ai/u/api-keys>. Choose an expiration and the workspace context the key should act in, generate the key, and copy the token. Send it as a bearer token in the `Authorization` header when your backend calls a workflow endpoint. Treat the key like a password: keep it in your `.env` file, never commit it, and never paste it into the frontend, since anything in the browser is public.

The screenshot shows the 'API Keys' management page. At the top, it says 'Generate and manage API keys for the workflow service. View other keys'. Below this is a 'Workflow API Keys' section with a sub-header 'Generate API keys to authenticate with the workflow service. Use these keys in the `Authorization: Bearer` header when calling workflow endpoints.' There are two dropdown menus: 'Expiration' set to 'No expiry' and 'Workspace Context' set to 'Personal Workspace'. A 'Generate API Key' button is next to the 'Expiration' dropdown. Below this is a section titled 'Your API Keys' which is currently empty. At the bottom, a note says: 'Generate a Workflow API key at auto.supervity.ai/u/api-keys, then use it as a bearer token from your backend.'

6.4 **Use the platform documentation.** The full Auto documentation, including the workflow endpoints your backend will call, lives at <https://auto.supervity.ai/docs>. Read it before you wire anything; it is the authoritative reference for calling an Operator from code, and it will save you the most time of anything in this guide.

6.5 **Persist what the agent does.** Runs, decisions, policy evaluations, and exceptions should be written to the database so the dashboard, the audit trail, and the Insights layer have real history to work with. The template's audit-logging middleware is already there; use it.

6.6 **Fail safely.** If an Operator fails or a field is missing, the agent pauses and routes the item to the Workbench. It does not crash, and it does not invent a value.

7. The Command Center surfaces, explained

7.1 The dashboard. The live operational picture for your track: the headline metrics, agent status, open exception queues, and whatever a leader in that role would check first each morning. The numbers must move as the agent works.

7.2 The AI Policies engine. The rules a business owns. In Round 1 you were told not to lean on a prebuilt policy module and to own the logic yourself; Round 2 is where you build that ownership properly, as a working engine in your Command Center. At least three active policies that genuinely constrain what the agent may do alone, expressed as deterministic rules or natural language, and evaluated **before** an action executes rather than reported afterwards. A business user must be able to change a threshold or rule in the interface, with no code, and see the agent behave differently on the next run. Every evaluation is logged so a decision can be traced later. Typical policies are approval bands, tolerances, eligibility rules, routing rules, and the conditions that force a freeze or an escalation.

7.3 The AI Insights engine. The patterns a person would never assemble by hand. Insights must be computed from the data your agent actually processed, carry a severity, and end in a clear action path. Two kinds are worth building: **operational insights** about the work itself, such as recurring anomalies, risk clusters, and forecasts; and **automation-opportunity insights** about the operation, such as a manual step that keeps recurring and could become a policy or a new Operator. Static charts and the template's seeded demo data do not count.

7.4 The AI Manager. The conversational surface over the operation. A person asks a question in plain language and gets an answer grounded in real records, and can trigger or re-trigger an Operator from the same place. It should never invent an answer it cannot support from the data.

7.5 The Data Manager. A live registry of every system the build is connected to, what each is used for, and whether the connection is healthy. It is how a judge sees that your integrations are real.

7.6 The Workbench. The human queue. Every item arrives with full context and the agent's recommendation. A person approves, modifies, or rejects, the decision is recorded, and the workflow continues from there. If a human correction also changes future behavior, that is self-learning and it earns bonus points.

8. Bring your own systems

8.1 Teams provision their own integrations. Create your own accounts on the CRMs, ticket tools, databases, and applications you need, and connect them either through a native Auto integration or through a code-built API Operator.

8.2 Every build needs **at least three live integrations across at least two categories**, including at least one channel and at least one system of record, all visible and healthy in the Data Manager.

8.3 Google integrations (Drive, Sheets, Gmail, Calendar) are in beta for this event. Use alternatives such as Airtable, Supabase, SharePoint, OneDrive, Box, Dropbox, and Outlook.

8.4 Integrations must carry real data. An integration that is connected but unused, or a Data Manager entry that is hardcoded, does not count toward the floor.

9. The dataset

9.1 Round 2 ships an expanded data pack per track. It keeps the Round 1 schema, so your existing build still reads it, and adds new tables, new columns, more volume, and harder cases so the wider operation has real work to do.

9.2 The dataset is how you exercise your integrations, not a file to read off disk. Use it one of two ways. Either **seed** your connected systems with it, loading the records into the CRM, ticket tool, database, or base you stand up, or **serve** it from a store such as SharePoint, OneDrive, Box, or Dropbox and point an integration at it. In both cases the AI Employee pulls that data back through the live integration and acts on it.

9.3 Do not hardcode to the rows you are given. A judge may ask you to run a record you did not prepare, or ask how the build behaves when the data changes, so the logic needs to hold for the general case rather than for a handful of chosen rows.

9.4 Datasets provided by Supervity are for use in this hackathon only and may not be redistributed.

10. Timeline, build, and submission

10.1 The Round 2 brief and the template repository are released on 25 July. The preparation window is 26 to 27 July: set up your environment, clone the template, get it running, and plan your Operators.

10.2 The remote build phase runs **3 to 7 August**. This is the five-day build.

10.3 **Day 1, Saturday 8 August, offline build at Asia Pacific University, Kuala Lumpur.** Check-in opens at **10:00 AM**; be on-site between **10:00 and 10:45** for check-in and breakfast. All teams should be present in person. The **code freeze** is at **11:59 PM** the same day, and no commits are accepted after it.

Time	Activity
10:00 to 10:45	Arrival, check-in and breakfast
10:45 to 11:00	Morning briefing and day overview
11:00 to 13:00	Build session, part one
13:00 to 14:00	Lunch break
14:00 to 16:30	Build session, part two
16:30 to 17:00	Office hours and final Q&A
17:00 to 17:30	Prep and rehearsal, showcase run-throughs
17:30 to 19:00	Build session, part three
19:00	Closing for the day
23:59	Code freeze. Whatever is deployed at this point is what is judged

10.4 **Day 2, Sunday 9 August, Grand Finale at APU.** Arrive between **10:00 and 10:45** again for check-in and breakfast. You have fifteen minutes to set up before judging begins at 11:00, so bring everything you need to run your build.

Time	Activity
10:00 to 10:45	Arrival, check-in and breakfast
10:45 to 11:00	Team set-up
11:00 to 13:00	Judging, track showcase, part one
13:00 to 14:00	Lunch break
14:00 to 16:30	Judging, track showcase, part two
16:30 to 17:00	Judges complete scoring
17:00 to 17:30	Keynote from the Supervity team, followed by a short address from the judges
17:30 to 19:00	Prize distribution ceremony
19:00	Closing

Your showcase slot is **10 to 12 minutes**: roughly 8 to 10 minutes to demonstrate your build live, and 2 to 3 minutes of questions from the judges. Your track's running order is confirmed on the morning of Day 2 and posted in Discord.

10.5 A Round 2 submission consists of: the team name and members; the assigned track; a link to the team's Auto workspace so judges can verify the Orchestrator and Operators are real; the repository containing the coded Command Center; a running instance judges can reach during the demo; and a short note stating the outcome metric the build moves and the integrations it uses.

10.6 Your build must run from a clean clone. Judges will not debug your machine. Make sure the Command Center comes up, the agent is reachable, and the demo path works end to end before the freeze.

11. How your build is judged

11.1 The qualification gate. Judges satisfy themselves that the build clears four business conditions before scoring it. This is a judgement call, not a technical checklist, and it exists so that a build which does not actually address the business problem is not scored against ones that do.

1. **It solves the business problem it was given.** It does the job of the role it claims to automate, end to end, rather than one interesting fragment of it.
2. **A human is genuinely in the loop.** When the agent should not decide alone, the work reaches a person with enough context to decide, and that decision completes the workflow.
3. **It is connected to real systems.** It works against live integrations a business would actually use, not a closed demo talking to itself.
4. **It ties to the real world and works live.** It is an improvement on how the work is done today, and you can run it and explain it in front of a judge.

11.2 Round 2 requirements. Separately from the gate, these are the things your build must contain. They are not optional, and the rubric assumes them: an Orchestrator on Auto coordinating **at least five distinct Operators**; a Command Center wired to that agent through the backend API, showing live activity rather than the template's demo data; **at least three AI Policies**, editable without code and applied before the agent acts; AI Insights generated from the data your agent processed; **at least three live integrations** across two categories, including one channel and one system of record, visible in the Data Manager; and a Workbench where a real exception is cleared by a human.

11.3 The core rubric. Past the gate, five criteria carry the score, weighted to 100.

Round 1 scored business output at 40 with three supporting lines. Round 2 lowers it to 30 and adds lines for Policies and Insights, because this round is about the governed operation around the agent, not the output alone. Nothing you were rewarded for in Round 1 stops mattering; more things now matter alongside it.

Criterion	Weight	Point split
Business output	30	Solves the core business problem 15 · quantified metric movement 10 · sensible handling of exceptions and edge cases 5
Architecture on Auto (agent and integrations)	20	Decomposition into five or more real Operators 8 · orchestration depth 7 · integration realism and Data Manager 5
Customizability and Policies	20	Applied before the agent acts 8 · live no-code configurability 7 · auditability and breadth 5
AI Insights	15	Generated from real processed data 6 · non-trivial and correct with severity 5 · actionable next step 4
Command Center and live demo	15	Live dashboard 6 · grounded AI Manager 4 · coherent end-to-end run 5
Total	100	Bonus is additive on top, up to +10

11.4 Bonus points, additive, up to +10. Bonus rewards going beyond the brief once the stated problem is solved: extra Operators and richer downstream actions; forecasting; self-learning, where a human correction or override at the Workbench is captured and changes future behavior; deeper auditability and governance, so a business user can trace every decision without an engineer; meaningful use of open-source

components, models, libraries, or frameworks that are named and given a real role rather than a token import; and genuine innovativeness and creativity on top of the problem statement.

11.5 Judged live. Every build is judged on a live run in front of the panel, and judges score what they see happen rather than the slides. Presentation polish is not a scoring dimension. Expect to be asked to run a case you did not rehearse, and to explain your own architecture.

11.6 Per-track enterprise judges score their own domain, so a finance leader scores Finance and an operations leader scores Operations. Judges' decisions are final.

12. Originality, conduct, and support

12.1 All work must be created during the hackathon window by the registered team. Reusing your own Round 1 build is expected and encouraged. Passing off another team's or a third party's work as original is not allowed.

12.2 Building with AI assistance is expected; Auto generates code. What is judged is the resulting AI Employee and the operation around it: its decomposition, its governance, its exception handling, and its business output.

12.3 Teams retain ownership of their intellectual property. By entering, teams grant Supervity a licence to feature the submission in case studies, recordings, and promotional material with attribution.

12.4 All participants follow a standard code of conduct: professional, respectful, and inclusive behavior at all times, on Discord, in every official channel, in submissions, and in person at APU. Harassing, discriminatory, abusive, or demeaning behavior toward organizers, judges, participants, or anyone else results in points being deducted or the team being disqualified, depending on severity. Serious or repeated violations mean immediate disqualification.

12.5 No plagiarism, no misrepresentation of a build, and no tampering with another team's systems or submissions. A submission that fakes a demo, hardcodes to sample data, or misrepresents what the AI Employee actually does live will be disqualified when the discrepancy surfaces during judging.

12.6 Daily office hours run in Discord throughout the build, and every official ruling is posted and pinned so all teams see the same answer.

13. Frequently asked questions

Round 2 in general

What is the goal of Round 2, plainly? To turn your Round 1 agent into an operation someone could run. More Operators doing more of the real job, a Command Center that shows what is happening, policies that constrain the agent, insights that surface what a person would miss, and a human queue for the decisions the agent should not make alone.

How is Round 2 different from Round 1? Round 1 tested decomposition: could you split a problem into an Orchestrator and Operators and connect real systems. Round 2 tests governance and completeness: does the agent cover more of the business function, does a person control it through policies, is every decision visible, and can a human step in cleanly.

Do I have to rebuild my Round 1 agent? No, and you should not. Keep your Round 1 Orchestrator and Operators, extend them to at least five Operators, and connect them to the Command Center. Building fresh agents on Auto is allowed if you prefer.

Does my track change? No. You keep the track you were assigned in Round 1, and team composition stays the same.

How long is the build? The remote build runs 3 to 7 August, five days. The offline build is 8 August at APU with a code freeze at 11:59 PM, and the Grand Finale is 9 August.

What does a finished Round 2 build look like? A trigger arrives through a live integration, your Orchestrator on Auto calls five or more Operators, a policy you can edit in the interface decides what executes automatically, an exception lands in the Workbench and a person clears it, the dashboard and insights update from what actually happened, and the Data Manager shows every connected system healthy.

The agent, Orchestrator, and Operators

What exactly is an Operator, and what is an Orchestrator? An Operator does one job end to end. An Orchestrator runs other Operators, deciding order, passing context, retrying, and escalating. The Orchestrator does not do the detailed work itself.

How do I call an Operator from inside an Orchestrator? Name it in plain language inside the Orchestrator's workflow, for example "I want to trigger the Risk Screen Operator Agent" or "call the Extraction operator agent." Auto runs that Operator as a step inside the Orchestrator.

How many Operators do I need? At least five distinct Operators under one Orchestrator. That is a gate condition, not a preference.

How do I know my Operators are genuinely distinct? Delete one on paper. If a specific business capability disappears, it was real. If nothing changes, you split one agent for appearances.

Must all orchestration be on Auto? Yes. The Orchestrator and Operators must be built on Supervity Auto. The Command Center, Policies, Insights, Manager, Data Manager, and Workbench are coded on the template and may use any model.

Can a single agent solve this? No. A single mega-agent fails the gate regardless of how well it performs.

Can my Orchestrator run Operators in parallel? Yes, and you should where the work allows it. Parallel fan-out and fan-in, conditional branching, retries, and escalation are exactly what the architecture score looks for.

The template and setup

Where is the template, and do I have to use it? <https://github.com/digitamizers/AutoPilot-Template>, and yes. Every Round 2 build starts there so judges can run every build the same way.

What do I need installed? Docker Desktop and Git. On Windows, WSL 2 must be enabled.

How do I start it? Clone the repository, copy `.env.example` to `.env`, start Docker Desktop, then run `make up` on macOS or Linux or `.\scripts\start.ps1` on Windows. The dashboard is at <http://localhost:3001> and the API docs at <http://localhost:8001/api/docs>.

The first run is slow. Is that normal? Yes. The first build downloads images and takes two to five minutes. Later starts take about fifteen seconds.

Port 3001 or 5432 is already in use. Something else is on that port, often a local PostgreSQL. Stop it, or change the port in `docker-compose.yml`.

The frontend is blank. The backend is probably not healthy. Check `curl http://localhost:8001/api/health` and read `docker compose logs backend`. Crash-looping containers are usually a missing environment variable or a database issue.

How does my backend authenticate to Auto? With a Workflow API key you generate at <https://auto.supervity.ai/u/api-keys>. Pick an expiration and workspace context, generate it, and send it as a bearer token in the `Authorization` header from your backend. Keep it in `.env`, never in the frontend.

Where is the platform documentation? <https://auto.supervity.ai/docs>. It is the authoritative reference for the workflow endpoints your backend calls.

Does the template's own backend count as one of my three integrations? No. The backend is part of your build. The three integrations are external systems your Operators read from and write to, such as a channel, a CRM, a ticket tool, or a database.

Do I need to set up authentication? No. The template ships with a development bypass so it starts with a session in place. Real authentication is a bonus, not a requirement.

Can I change the template's design? Yes, and you should make it yours, but keep it looking like a Supervity product. The repository includes a design-system reference.

Can I add my own database tables and endpoints? Yes. Alembic migrations are already set up. Persisting runs, decisions, and exceptions is expected.

Policies, Insights, Manager, Workbench

What counts as a working AI Policies engine? At least three active policies that genuinely constrain the agent and are evaluated before an action executes. A business user must be able to change a threshold or rule in the interface, with no code, and see behavior change. Every evaluation must be logged.

How will judges test my policies? By changing one in front of you. Expect a judge to edit a threshold in your Command Center, ask you to re-run, and check that the agent behaved differently and that the evaluation was recorded.

Can policies be natural language, or must they be rules? Either. Deterministic rules or a model evaluating an Operator's output against a plain-language rule both count, as long as the policy actually gates the action and is editable without code.

What counts as a real insight? An observation computed from the data your agent actually processed, with a severity and a clear action path. Recurring patterns, anomalies, risk clusters, bottlenecks, and forecasts all

count. A static chart or the template's seeded demo data does not.

Can insights suggest automation opportunities? Yes, and that is one of the strongest things to build. An insight that spots a manual step recurring often enough to deserve a policy or a new Operator is exactly the intent.

What should the AI Manager do? Answer questions about the operation from real records, and let a person trigger or re-trigger an Operator. It should not be a general chatbot, and it must not invent answers.

What belongs in the Workbench? Any decision the agent should not make alone. Each item arrives with full context and the agent's recommendation, a person approves, modifies, or rejects, and the decision is recorded.

Does a human correction have to change future behavior? Not to pass, but it earns bonus points. Capturing an override and adapting later behavior is the self-learning bonus.

Data and integrations

Is the Round 2 dataset the same as Round 1? It keeps the Round 1 schema and expands it. Each track gains new tables, new columns, more volume, and harder cases so the wider operation has real work to do.

Can I just read the spreadsheet from disk? No. Either seed your connected systems with it, or serve it from SharePoint, OneDrive, Box, or Dropbox and point an integration at it, then pull it back through the live connection.

How many integrations do I need? At least three across at least two categories, including one channel and one system of record, all visible and healthy in the Data Manager.

Can I use Google Drive, Sheets, Gmail, or Calendar? Google integrations are in beta for this event. Use Airtable, Supabase, SharePoint, OneDrive, Box, Dropbox, or Outlook instead.

Will judges bring their own data? No. You demonstrate on your own working setup. A judge may ask you to process a record you did not prepare in advance, and will ask how the build behaves when the data or the rules change, so avoid tuning your logic to a handful of chosen rows.

Judging, submission, and the finale

How is Round 2 scored? A pass-or-fail gate first, then five criteria weighted to 100: Business output 30, Architecture on Auto 20, Customizability and Policies 20, AI Insights 15, and Command Center and live demo 15, with bonus additive up to 10.

What earns bonus points? Extra Operators and richer downstream actions, forecasting, self-learning from Workbench overrides, deeper auditability, meaningful open-source use, and genuine creativity beyond the brief.

Does a polished interface win? No. Presentation polish is not a scoring dimension. Judges score the trace, not the slides.

What do I submit? Team name and members, your assigned track, a link to your Auto workspace, the repository with your Command Center, a running instance judges can reach during the demo, and a short note on the outcome metric and integrations.

Does my build have to run from a clean clone? Yes. Judges will not debug your machine. Test it before the freeze.

What are the most common ways teams lose points? Shipping the template's demo data on the Policies or Insights pages; a dashboard that does not move when the agent runs; policies that display but do not

gate anything; an exception path that was never demonstrated live; and hardcoding to a few chosen rows so nothing works when a judge asks for a different case.

Where do I ask questions during the build? The official Discord channels. Daily office hours run there, and every official ruling is posted and pinned so all teams see the same answer.

Not apps. Not agents. AI Employees run the Autos.